Step 1: Create Restaurant Model

Open:

```
api/models.py
```

Add:

```
from django.db import models


class Restaurant(models.Model):
    name = models.CharField(max_length=100)
    cuisine = models.CharField(max_length=100)
    rating = models.FloatField()

    def __str__(self):
        return self.name
```

Now run migrations:

```
python manage.py makemigrations
python manage.py migrate
```

Register Model in Admin Panel

Open:

```
api/admin.py
```

Add:

```
from django.contrib import admin
from .models import Restaurant

admin.site.register(Restaurant)
```

Now create admin user:

```
python manage.py createsuperuser
```

Run server:

```
python manage.py runserver
```

Open admin panel:

```
http://127.0.0.1:8000/admin/
```

Step 2: Create ModelSerializer

Open:

```
api/serializers.py
```

Add:

```
from rest_framework import serializers
from .models import Restaurant


class RestaurantSerializer(serializers.ModelSerializer):

    class Meta:
        model = Restaurant
        fields = '__all__'
```

This serializer converts Restaurant objects into JSON and also validates incoming data.

Step 3: Create CRUD API Using APIView

Open:

api/views.py

Add:

```
from rest_framework.views import APIView
from rest_framework.response import Response
from rest_framework import status

from .models import Restaurant
from .serializers import RestaurantSerializer


# GET all restaurants
# POST new restaurant

class RestaurantListCreateAPIView(APIView):

    def get(self, request):
        restaurants = Restaurant.objects.all()
        serializer = RestaurantSerializer(restaurants, many=True)

        return Response(
            serializer.data,
            status=status.HTTP_200_OK
        )

    def post(self, request):
        serializer = RestaurantSerializer(data=request.data)

        if serializer.is_valid():
            serializer.save()

            return Response(
                serializer.data,
                status=status.HTTP_201_CREATED
            )

        return Response(
            serializer.errors,
            status=status.HTTP_400_BAD_REQUEST
        )


# PUT/PATCH/DELETE by ID

class RestaurantDetailAPIView(APIView):
```

```
    def get_object(self, id):
        try:
            return Restaurant.objects.get(id=id)
        except Restaurant.DoesNotExist:
            return None

    def put(self, request, id):
        restaurant = self.get_object(id)

        if not restaurant:
            return Response(
                {"error": "Restaurant not found"},
                status=status.HTTP_404_NOT_FOUND
            )

        serializer = RestaurantSerializer(
            restaurant,
            data=request.data
        )

        if serializer.is_valid():
            serializer.save()

            return Response(
                serializer.data,
                status=status.HTTP_200_OK
            )

        return Response(
            serializer.errors,
            status=status.HTTP_400_BAD_REQUEST
        )

    def patch(self, request, id):
        restaurant = self.get_object(id)

        if not restaurant:
            return Response(
                {"error": "Restaurant not found"},
                status=status.HTTP_404_NOT_FOUND
            )

        serializer = RestaurantSerializer(
            restaurant,
            data=request.data,
            partial=True
        )
```

```
        if serializer.is_valid():
            serializer.save()

            return Response(
                serializer.data,
                status=status.HTTP_200_OK
            )

        return Response(
            serializer.errors,
            status=status.HTTP_400_BAD_REQUEST
        )

    def delete(self, request, id):
        restaurant = self.get_object(id)

        if not restaurant:
            return Response(
                {"error": "Restaurant not found"},
                status=status.HTTP_404_NOT_FOUND
            )

        restaurant.delete()

        return Response(
            {"message": "Restaurant deleted successfully"},
            status=status.HTTP_200_OK
        )
```

Step 4: Configure URLs

Open:

```
api/urls.py
```

Add:

```
from django.urls import path
from .views import (
    RestaurantListCreateAPIView,
    RestaurantDetailAPIView
)
```

```
urlpatterns = [

    # GET all & POST
    path(
        'restaurants/',
        RestaurantListCreateAPIView.as_view()
    ),

    # PUT PATCH DELETE
    path(
        'restaurants/<int:id>/',
        RestaurantDetailAPIView.as_view()
    ),
]
```

Now include api URLs inside:

```
foodiehub/urls.py
```

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('api/', include('api.urls')),
]
```

API Endpoints

Create Restaurant

POST

```
http://127.0.0.1:8000/api/restaurants/
```

Body:

```
{
    "name": "Pizza Hub",
```

```
    "cuisine": "Italian",
    "rating": 4.5
}
```

Get All Restaurants

GET

```
http://127.0.0.1:8000/api/restaurants/
```

Update Restaurant

PUT

```
http://127.0.0.1:8000/api/restaurants/1/
```

Partial Update

PATCH

```
http://127.0.0.1:8000/api/restaurants/1/
```

Delete Restaurant

DELETE

```
http://127.0.0.1:8000/api/restaurants/1/
```

Step 5: Refactor Using GenericAPIView + Mixins

Now we reduce boilerplate code.

Open:

```
api/views.py
```

Add:

```
from rest_framework import mixins, generics

from .models import Restaurant
from .serializers import RestaurantSerializer


# LIST + CREATE

class RestaurantListMixinAPIView(
    mixins.ListModelMixin,
    mixins.CreateModelMixin,
    generics.GenericAPIView
):

    queryset = Restaurant.objects.all()
    serializer_class = RestaurantSerializer

    def get(self, request):
        return self.list(request)

    def post(self, request):
        return self.create(request)


# UPDATE + DELETE

class RestaurantDetailMixinAPIView(
    mixins.UpdateModelMixin,
    mixins.DestroyModelMixin,
    mixins.RetrieveModelMixin,
    generics.GenericAPIView
):

    queryset = Restaurant.objects.all()
    serializer_class = RestaurantSerializer
    lookup_field = 'id'

    def get(self, request, id):
        return self.retrieve(request, id=id)
```

```
    def put(self, request, id):
        return self.update(request, id=id)

    def patch(self, request, id):
        return self.partial_update(request, id=id)

    def delete(self, request, id):
        return self.destroy(request, id=id)
```

Update URLs for Mixins Version

```
from django.urls import path
from .views import (
    RestaurantListMixinAPIView,
    RestaurantDetailMixinAPIView
)

urlpatterns = [

    path(
        'restaurants/',
        RestaurantListMixinAPIView.as_view()
    ),

    path(
        'restaurants/<int:id>/',
        RestaurantDetailMixinAPIView.as_view()
    ),
]
```